

1SIMATS ENGINEERING

ASSESSMENT TOOL 1 – ANALYTICAL PROBLEM SOLVING

Course: Artificial Intelligence (CSA17)

Course Outcome: CO1 – Apply AI basics such as intelligent agents and uninformed search methods to solve problems effectively.

Assessment Weightage: 35% | Total Marks: 50

Name	SenthmiazhSelven K
Register No.	192321143
Date	6.10.26

1. Water Jug Problem

Problem: We are given a 4-gallon jug and a 3-gallon jug, with no measuring marks. Using a water pump and the allowed pouring operations, obtain exactly 2 gallons of water in the 4-gallon jug.

Initial state: (0, 0), where the first value represents the amount in the 4-gallon jug and the second represents the amount in the 3-gallon jug.

Goal state: (2, 0) or any state in which the 4-gallon jug contains exactly 2 gallons.

1. Fill the 4-gallon jug. State: (4, 0).
2. Pour water from the 4-gallon jug into the 3-gallon jug until the 3-gallon jug is full. State: (1, 3).
3. Empty the 3-gallon jug onto the ground. State: (1, 0).
4. Pour the 1 gallon from the 4-gallon jug into the 3-gallon jug. State: (0, 1).
5. Fill the 4-gallon jug again. State: (4, 1).
6. Pour from the 4-gallon jug into the 3-gallon jug until the 3-gallon jug is full. It needs 2 more gallons, so 2 gallons remain in the 4-gallon jug. State: (2, 3).
7. Empty the 3-gallon jug if required. The 4-gallon jug already contains exactly 2 gallons.

Final answer: Exactly 2 gallons are obtained in the 4-gallon jug. The solution demonstrates state-space problem solving, where each pair (x, y) represents the quantities of water in the two jugs.

2. Mars Rover as an Intelligent Agent

A Mars Rover is an intelligent agent that senses its surroundings, makes decisions, moves across the Martian surface, collects scientific data and samples, and communicates useful information to Earth.

i. Percepts of the agent:

- Images and terrain information from cameras.
- Distance, obstacles and surface information from sensors.

- Temperature, atmospheric and environmental measurements.
- Chemical and mineral information from scientific instruments.
- Status information such as battery level, wheel condition and equipment state.
- Commands or instructions received from mission control.

ii. Operating environment:

The Mars Rover operates in a partially observable, dynamic and uncertain environment. It has limited communication with Earth, limited energy, rough and unknown terrain, environmental hazards, and physical constraints. The environment can be described as partially observable, sequential, stochastic, dynamic, continuous and mostly single-agent.

iii. Actions the agent can take:

- Move forward, backward or turn.
- Navigate around obstacles.
- Use cameras and scientific sensors.
- Collect and analyze soil or rock samples.
- Take photographs and measurements.
- Store scientific data.
- Communicate information to mission control.
- Manage power and select safe operating actions.

iv. Performance evaluation:

The rover can be evaluated by the quality and quantity of scientific data collected, successful sample analysis, distance or areas explored, safe navigation, number of objectives completed, energy efficiency, equipment reliability, and successful communication with Earth.

v. Suitable agent architecture:

A model-based, goal-based and utility-based agent architecture is most suitable. A model-based component is needed because the rover cannot observe the entire environment. Goal-based reasoning helps it achieve mission objectives such as reaching a location or collecting a sample. Utility-based decision-making helps it select safer and more valuable actions when several alternatives are available.

3. 8-Queens Problem – Search Space and Problem Formulation

Problem: Place eight queens on an 8×8 chessboard so that no two queens attack each other. Two queens must not be in the same row, column or diagonal.

Problem formulation:

- Initial state: An empty chessboard.
- State: A partial configuration of queens placed on the board without violating the constraints.

- Actions: Place a queen in a valid position in the next column (or another selected unoccupied position) such that it does not conflict with previously placed queens.
- Transition model: The selected position is occupied by a queen, producing a new partial configuration.
- Goal test: Eight queens are placed and no pair of queens attacks each other.
- Path cost: Usually not important for the standard 8-Queens problem; each placement may be assigned a unit cost if a search algorithm requires a path-cost definition.

Search-space representation:

A convenient representation is $[r_1, r_2, r_3, r_4, r_5, r_6, r_7, r_8]$, where r_i indicates the row occupied by the queen in column i . A valid solution must contain no repeated row and must satisfy the diagonal condition $|r_i - r_j| \neq |i - j|$ for every pair of columns i and j .

Search strategy:

Backtracking depth-first search can systematically explore the search space. A queen is placed column by column. If a placement creates a conflict, that branch is abandoned and the algorithm backtracks to try another position. This avoids exploring complete configurations that are already known to be invalid.

4. OLA Cab Booking – Problem-Solving Agent and Pseudocode

The suitable problem-solving agent is a goal-based problem-solving agent. The goal is to travel from the customer's source location to the destination. The agent considers available cab types, route, estimated travel time, fare, availability and the customer's selected preference before making a booking.

PEAS description:

- Performance: Reach the destination safely, efficiently and according to the customer's preference, while considering fare and travel time.
- Environment: Roads, traffic, source and destination locations, available drivers and available cab types.
- Actuators: Select cab type, choose route, confirm booking and provide navigation instructions.
- Sensors/Input: GPS location, destination, traffic information, cab availability, fare and customer preferences.

Pseudocode:

```
START
  Get customer's current location
  Get destination
  Display available cab types
  Get customer's preferred cab type

  For each available cab:
    Check availability
```

```

    Estimate fare
    Estimate travel time
    Check route and traffic conditions

    Select cab according to customer preference
    If preferred cab is unavailable:
        Show suitable available alternatives

    Display selected cab, fare and estimated time
    Confirm booking

    If booking is confirmed:
        Assign cab
        Start navigation
        Track trip until destination is reached
        Complete trip
    Else:
        Cancel booking

```

END

5. Uniform Cost Search (UCS) for Least-Cost Delivery Path

Problem: A logistics company wants to transport packages from starting warehouse S to destination warehouse G at minimum total cost. Each edge in the delivery network has a travel cost representing factors such as fuel and time.

Uniform Cost Search selects and expands the frontier node having the smallest cumulative path cost $g(n)$. It is appropriate for this problem because the objective is to find the least-cost path.

UCS procedure:

1. Start with warehouse S in the priority queue with cost 0.
2. Remove the node with the lowest cumulative cost from the priority queue.
3. If the removed node is G, return its path as the least-cost solution.
4. Otherwise, expand the node and calculate the new cumulative cost for each neighboring warehouse.
5. Insert or update the neighboring nodes in the priority queue when a cheaper path is found.
6. Repeat until G is selected for expansion.

Note: The uploaded assessment page contains the description of the weighted graph but the actual graph/edge-cost figure is not present in the extracted four-page content. Therefore, a specific numerical least-cost path from S to G cannot be calculated from the supplied source without inventing missing edge costs.

UCS pseudocode:

```

UCS(start, goal):
    frontier = priority queue ordered by path cost
    frontier.push(start, 0)
    explored = empty set

```

```

cost[start] = 0
parent[start] = NULL

while frontier is not empty:
    current = frontier.pop_lowest_cost()

    if current == goal:
        return reconstruct_path(parent, goal), cost[goal]

    if current in explored:
        continue

    add current to explored

    for each neighbor of current:
        new_cost = cost[current] + edge_cost(current, neighbor)

        if neighbor not in cost OR new_cost < cost[neighbor]:
            cost[neighbor] = new_cost
            parent[neighbor] = current
            frontier.push(neighbor, new_cost)

return "No path found"

```

Conclusion

The five problems demonstrate important Artificial Intelligence concepts including state-space representation, problem formulation, intelligent-agent design, goal-based problem solving, backtracking search, and Uniform Cost Search. These methods allow an AI system to represent a problem clearly, explore possible solutions systematically, and select an appropriate solution based on the given objective.

Student Declaration

I certify that I have prepared this assignment based on the given assessment questions and that I will review and understand the solutions before submission.